Rendering the macOS Liquid Glass Appearance in 4D

By Anouar Moustarih, Quality Support Engineer, 4D Morocco

Technical Note 26-10

Table of Contents

Table of Contents 2
Abstract 4
Introduction 4
Prerequisites and requirements 4
  Software 4
  Declaring the component 5
  What is not required 5
How 4D applies a form's appearance 5
  Three behaviours to design around 5
    Substitution 5
    Scope 6
    Inert declarations 6
The form style sheet 6
  Light and dark, in four blocks 6
  Color values: no alpha at any syntax 7
  Lengths are written without a unit 7
  Styling a list box to look like a macOS table 8
    Fill and row painting 8
  Repeating the same values four times 8
The form description file 9
  Where the shared properties live 9
  List box columns and the three required names 9
  Properties that belong to the column 10
When a form object reads its data 10
  Binding happens before the first event 10
  Column headers bind too 11
  The selection and the highlight 11
Translucency without an alpha channel 11
  Drawn entirely by form objects 12
  Blending by linear interpolation 12
  Storing the alpha, not the result 13
  A material requires content behind it 13
  Pre-computing a lens over a fixed background 14
    Blur radius in relation to detail spacing 14
  Saturation and contrast are one operation 14
  Where the technique stops, and what covers the rest 15
  The edge is cut after the blur 15
  One operation, recurring in four roles 16
  Contrast must be checked against the material 16
Responding to an appearance change 17
  A form resolves its style sheet once, when it opens 17
  Reopening as a strategy 17
Window chrome, and how to reproduce it 17
  The unified title bar 17
  Resizable split layouts 18
Results 18

Key Takeaways 21
Translucency is arithmetic 21
A fixed background can be solved once 21
Store the alpha, not the result 21
A material needs something behind it 21
Conclusion 21

Abstract

4D 21 Rx (feature releases) provide form style sheets, the `prefers-color-scheme` media feature and form themes. Together these determine how a form is styled and how it responds to the system appearance setting.

This note documents the conventions that govern them. It covers the form style sheet and the structure of the form description file. It also covers the sequence in which a form object reads the value it displays. It documents a method for producing translucent surfaces, specular edges and a refracted background, using the value forms a form style sheet accepts.

The material was produced using 21 R4 under macOS. The scope is the interface layer.

Introduction

The form layer in 4D is declarative. A window is described in a form file and a style sheet, and 4D draws the result.

4D reads that description once, when the form opens. Every property accepts a defined set of values. A value outside that set is not applied. In most cases, nothing is reported.

The description must therefore be exact. 4D applies the values it is given and does not correct a value that is wrong.

This note sets out the conventions that govern the description. Each is presented as what the developer writes, what 4D does with it, and what appears on screen.

The sections that follow describe how these are applied.

This note addresses 4D developers who build desktop interfaces for macOS. No prior knowledge of macOS design conventions is assumed. No knowledge of Swift or AppKit is required.

The reader is expected to be familiar with 4D project mode, form objects and form events. Familiarity with form style sheets is useful but not required.

Prerequisites and requirements

This section lists what the material in this note requires. Every item below was used to produce the results shown later.

Software

Table 1. The software the techniques in this note require.

Item	Version used	Why it is required
4D	21 R4	Form themes and the `form-theme` media feature arrive in v21 R3. The style sheet structure described here depends on both.

Item	Version used	Why it is required
4D SVG component	Retrieved from the 4D repository	Renders the background and the panel images. Supplies the two filter primitives the material work needs.
macOS	26, Tahoe 27, Golden Gate	Provides the appearance setting the style sheet responds to, and the reference for the conventions described.

Declaring the component

The SVG component is declared as a project dependency rather than copied into the project folder. The declaration lives in `Project/Sources/dependencies.json`.

```
{
   "dependencies": {
      "4D SVG": {
         "github": "4d/4D-SVG",
         "version": "latest"
      }
   }
}
```

4D resolves the dependency when the project opens. The component then supplies its commands to the project's own methods.

Two filter primitives carry the material work:

- A Gaussian blur softens the background.
- A color matrix adjusts saturation and contrast.

What is not required

- **No native plug-in.** The interface is built, styled and run inside 4D.
- **No external toolchain.** The background and panel images are produced by the project itself, through the SVG component.
- **No image editor.** Every image in the finished window is generated from the measured values.

How 4D applies a form's appearance

Most values 4D does not accept are ignored rather than reported. The window still opens and stays usable, which is what makes the three behaviours below easy to miss. Lengths behave differently and are covered under the style sheet.

Three behaviours to design around

Substitution

An object given a value 4D does not accept draws with its own default instead. The window is displayed and stays usable.

A picture path that does not resolve behaves this way. The picture object draws nothing, and whatever sits behind it is visible.

The practical step is to confirm that every file a form names is present in the built project. Checking the development machine alone is not enough.

Scope

The style sheet parser reads rules in sequence. A value it does not accept ends the reading at that point. Every rule already read stays in force, and every rule below it is never read at all.

The visible result is that styling applies to part of a design and not to the rest. Classes defined earlier in the file are unaffected.

Inert declarations

A property set to a value outside its list of permitted names is saved in the form file and has no effect on screen. The object looks correct and does nothing.

An event name outside the published list behaves this way. The object is complete in every other respect.

The form style sheet

4D applies a style sheet to form objects. The file is placed in the project's Sources folder. A file named `styleSheets.css` applies everywhere. A file named `styleSheets_mac.css` applies on macOS only.

The style sheet assigns properties to classes. A form object names a class through its `class` property. The property names are the same names used in the form description file, such as `fill`, `stroke` and `fontSize`.

Note: *A property set in the form editor overrides the style sheet for that object. A property intended to be themed should not also be set on the object.*

Light and dark, in four blocks

4D supports the `prefers-color-scheme` media feature. A developer can therefore define one set of values for light appearance and another for dark.

One further media feature exists on 4D v21 R3 and later. The `form-theme` feature identifies the active form theme, and a block that names it outranks a block that does not.

The consequence is that two blocks are not enough. A style sheet containing only a base block and a dark block renders unthemed on 21 R3 and 21 R4. A style sheet containing only the theme blocks renders unthemed on earlier builds.

Four blocks cover both. Figure 1 shows the order, which matters because the last matching block wins.

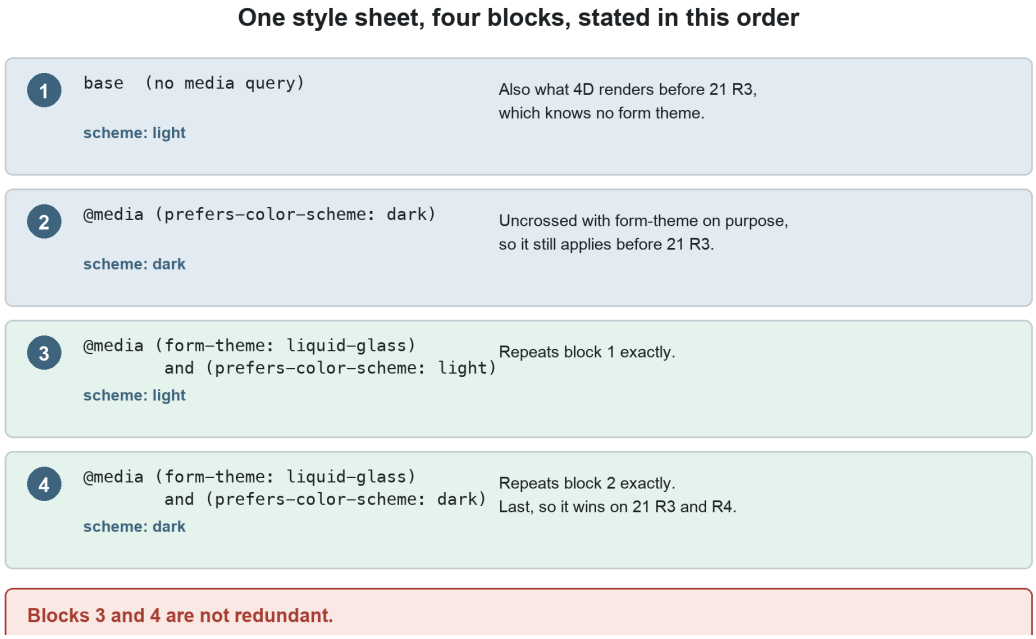


Figure 1. The four blocks of a form style sheet, and why the pair is duplicated.

Color values: no alpha at any syntax

The form style sheet parser accepts six-digit hexadecimal colors. It also accepts a short list of keywords, including `transparent`.

Every other color syntax is rejected. That includes `rgba()`, `hsla()`, eight-digit hexadecimal and three-digit hexadecimal.

Rejection is not reported. The parser discards the rule and every rule after it in the file. Classes defined earlier still work, which is what makes the fault so confusing.

The practical consequence is significant. macOS semantic label colors carry alpha by design. A developer copying those values directly will produce a style sheet that stops applying part-way down.

The solution is to blend the color in advance. The section on translucency describes the arithmetic.

Lengths are written without a unit

Lengths in a form style sheet carry no unit. A radius of ten is written as `10`.

This convention is enforced rather than ignored. The spelling `borderRadius: 10px` is reported as error 555 when the window opens.

An error that is reported is far easier to find than one that is not. This is the more helpful of the two behaviours.

Note: *A list box row height behaves the opposite way. In the form description file `rowHeight` is a string and does take px, as in `"24px"`. The asymmetry is easy to reverse by accident.*

Table 2. Value rules for the form style sheet, and the consequence of breaking them.

Property kind	Accepted	Rejected	Consequence of a rejected value
Color	`#RRGGBB`, `transparent`, `automatic`	`rgba()`, `hsla()`, `#RRGGBBAA`, `#RGB`	Parsing ends, and later rules do not apply
Length	A bare integer, such as `10`	`10px`, `10pt`, `1em`, `50%`	Error 555 is reported on opening
Font weight	`normal`, `bold`	Numeric weights	Rule discarded
List box row height	A string with `px`, in the form file	A bare number	Property rejected

Styling a list box to look like a macOS table

A list box accepts more style properties than a rectangle. Three of them decide whether it reads as a table or as a spreadsheet.

- `alternateFill` paints every other row. It produces the banded appearance of a macOS table.
- `horizontalLineStroke` draws a rule between rows. Setting it to `transparent` removes the grid.
- `verticalLineStroke` draws a rule between columns. A macOS table has none.

A macOS inset table separates its rows by the alternating fill alone. Leaving both line strokes visible produces a spreadsheet.

The opposite treatment suits a sidebar. A rule between rows and none between columns reads as a grouped list rather than a table.

Fill and row painting

A `fill` on a list box stops row painting. A developer who sets a fill to control the background loses `alternateFill` at the same time.

One exception applies. A list box that paints no rows has no row painting to retain.

A navigation list is the common case. A list box paints its own background by default, as any standard control does, which covers a panel placed behind it.

Setting `fill` to `transparent` there lets that panel show through. The translucency section relies on this.

Repeating the same values four times

A four-block style sheet states every value four times. Written by hand it offers four chances to mistype one value and no way to notice.

A wrong value in the fourth block is invisible until somebody runs the current 4D build in dark appearance. That may be long after the project ships.

The practical defence is to write the design values down once, then produce the four blocks from that single list. The blocks cannot disagree, because they are one list written out repeatedly.

The form description file

A 4D form is stored as `form.4DForm`, a JSON file. 4D ships a JSON Schema describing the format, inside the application bundle at `Resources/JSONSchemas/formsSchema.json`.

The schema answers most questions about which property belongs where. Three answers are not evident from the form file alone.

Where the shared properties live

Most object properties are defined per type. A list box defines `columns`. A button defines `defaultButton`.

Ten properties are shared by every object. They are not defined per type. They live together at `definitions.types.objectCommon`.

The shared set contains the properties used most often:

```
top      left      width      height
bottom   right     sizingX    sizingY
visibility class
```

A type definition reaches that set through a JSON reference, written as a path:

```
{ "$ref": "#/definitions/types/objectCommon" }
```

Note: *Anything reading this schema must follow the whole path. Matching only the last segment looks for `definitions.objectCommon`, which does not exist, and finds nothing. Position, size and class then appear to be invalid properties.*

List box columns and the three required names

A list box column carries three names. The column has a `name`. Its header object has a `name`. Its footer object has a `name`.

All three are required, including the footer name on a list box that shows no footer.

The form editor reads these names while applying the style sheet to the list box.

The three names are therefore not optional, and the form editor relies on them while styling the list box.

```
"columns": [
```

```
  {
    "name": "colName",
    "dataSource": "This.label",
    "dataSourceTypeHint": "text",
    "width": 197,
    "enterable": false,
    "header": { "name": "colNameH", "text": "Vessel" },
    "footer": { "name": "colNameF", "text": "" }
  }
]
```

A minimal list box column carrying all three required names.

Properties that belong to the column

Read-only behaviour belongs to the column. The property is `enterable`, and it is set to false on each column.

The setting belongs to the column rather than to the list box. Setting it once per column is what makes a display-only register read-only.

The symptom is mild and easy to dismiss. A click places a text cursor in a cell, and a keystroke edits a value the window only displays.

When a form object reads its data

A form object reads its bound value at a defined moment. That moment is the contract between the form and the code that prepares what it shows.

Binding happens before the first event

A collection list box reads its data source before the form's On Load event runs. An input reads its value at the same moment. A list box column header reads its text there too.

Values are therefore prepared before the form opens, not inside On Load. By On Load the objects have already read what they display.

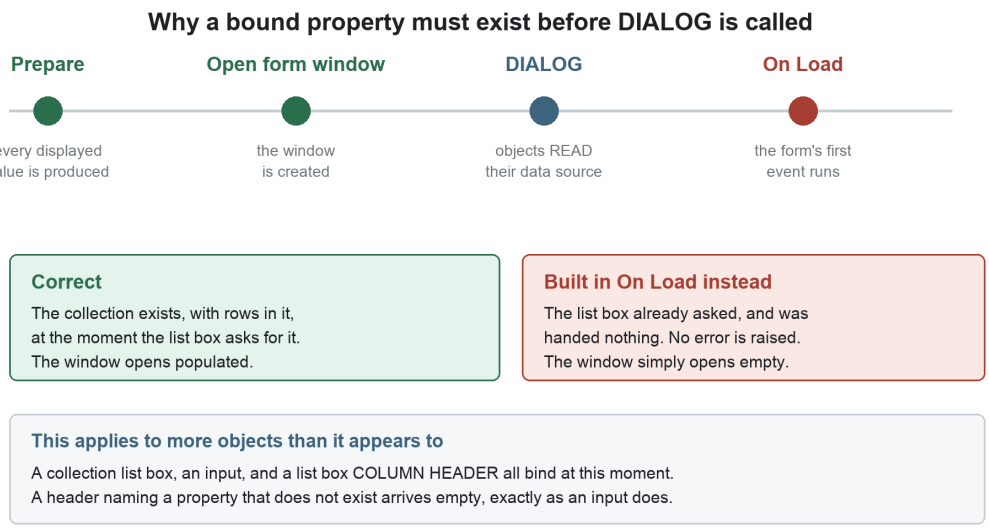


Figure 2. The binding moment, relative to the events available to the developer.

The reliable pattern prepares everything the form displays before **DIALOG** is called. An On Load handler written this way contains no data preparation at all.

Work that genuinely belongs in On Load is work that touches form objects. Setting a column title or hiding a picture cannot happen before the objects exist.

Column headers bind too

A header takes static text. A form expression placed there is not evaluated the way an input evaluates one, so the band is drawn without titles.

Note: *Static text is the value the header falls back to. A band retains its titles if the runtime call that adds a sort indicator is not made.*

The selection and the highlight

A list box bound to a collection loses its highlight if that collection is replaced.

Replacing a collection while the operator is clicking in it is the common case. The row is highlighted, the collection is replaced, and the highlight is lost before the mouse button is released.

Any control that then acts on the selected item receives nothing. The reported symptom is normally a control that does not respond.

A selection change is the one event that changes nothing about which rows are listed. Responding to it by rebuilding the list is both wasteful and harmful.

Translucency without an alpha channel

Modern macOS interfaces use translucent materials. A sidebar samples what is behind it, blurs it, and tints the result.

A form object draws the values it is given. It does not read the pixels of whatever sits behind the window, so it cannot sample them and blur them while it draws. The same appearance can be produced by computing the blur and the blend in advance.

Drawn entirely by form objects

Everything in this section is drawn by ordinary form objects. No web area takes part, and no HTML or browser engine is involved at any point.

Four facilities carry the whole result, and each is a standard part of 4D.

The form style sheet sets color and size. **The form theme and the prefers-color-scheme media feature** select the appearance. **A static picture object** displays an image. **The 4D SVG component** renders vector artwork into a picture.

What follows is arithmetic applied through those four. Nothing is added beside them and nothing is worked around.

Blending by linear interpolation

Translucency is a blend of two colors. If the environment forbids alpha but allows arbitrary colors, the blend can be computed in advance.

The formula is linear interpolation, applied to each channel:

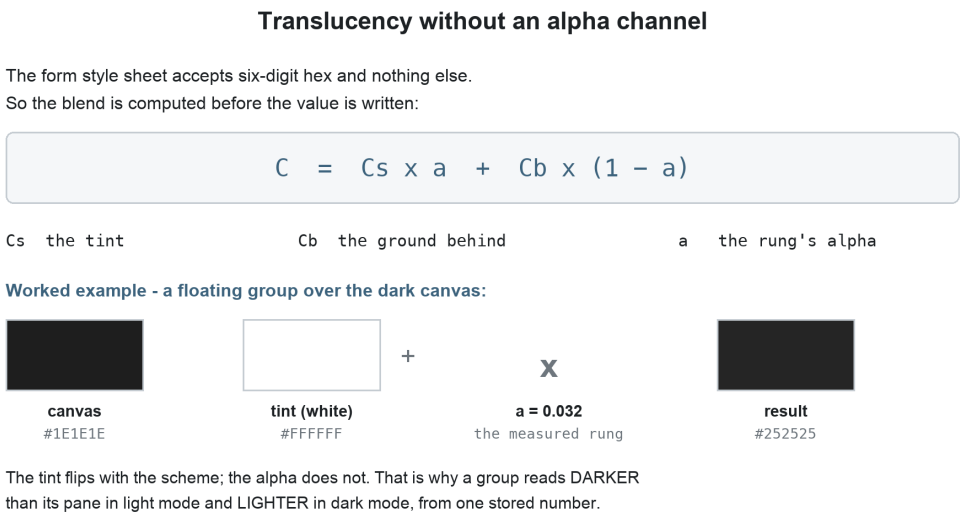


Figure 3. Computing an opaque color that matches a translucent one.

The result is an ordinary six-digit hexadecimal color. The style sheet never needs to know a material was involved.

A worked example makes the formula concrete. A panel sits on a white window background and carries a black tint at an alpha of 0.075.

Each channel is computed on its own. The red channel runs 255 + (0 - 255) x 0.075, which is 236.

Green and blue give the same answer, because both colors are neutral. The panel is therefore #ECECEC, and that value goes straight into the style sheet.

Table 4. One translucent surface, computed for both appearances.

Appearance	Window background	Tint	Alpha	Result
Light	#FFFFFF	#000000	0.075	#ECECEC
Dark	#1E1E1E	#FFFFFF	0.075	#2F2F2F

One alpha produces a darker panel in light appearance and a lighter one in dark. Only the tint changes, and it changes to the opposite extreme.

Storing the alpha, not the result

A design can store the finished colors or the alphas that produced them. Storing the alphas is more useful.

One stored alpha serves both appearances. The tint changes between light and dark. The alpha does not.

That single fact explains a detail of macOS design that often confuses developers. A raised surface is darker than its background in light appearance and lighter in dark appearance. Both come from one number, with the tint reversed.

A whole interface needs only a few of these numbers. Three cover every translucent surface in the example that follows.

Table 5. The three translucency steps, and where each one appears.

Rung	Alpha	Where it appears	Used for
chrome	0.075	The sidebar, over the content area	The navigation layer
glass	0.032	A grouped panel, over the pane behind it	A floating group
hairline	0.043	A divider, over the panel it sits on	A rule inside a group

Each row of that table is one step of translucency. Each is blended over the surface the object will sit on. A divider inside a panel is blended over the panel, not over the window behind it.

Using one hairline color everywhere is visibly wrong on one of the two surfaces.

A material requires content behind it

Translucent material has no color of its own. It filters what passes through it.

Over a flat background a material collapses into a plain rectangle. A design that requires a translucent surface must place content behind it to refract.

Pre-computing a lens over a fixed background

Blur, refraction and sampling are not operations a form object performs while it draws. They become unnecessary when the background cannot change.

If the background is a fixed image rather than a live surface, what sits behind each panel is known in advance. The optical work can be done once, before the project is built, and the result placed in the form as a picture.

The procedure produces one image per panel:

- Crop the background at the panel's exact rectangle.
- Displace the edges, so content bends as it passes under the rim.
- Blur the crop, which softens fine detail and keeps large structure.
- Raise saturation and contrast, because color under dense glass intensifies.
- Blend the tint at the chosen alpha.
- Draw the specular rim. That is a one-pixel light line along the top edge and a one-pixel dark line along the bottom. A raised glass edge catches light that way.
- Round the corners, filling outside them with the original background pixels.

The final step avoids relying on transparency. Filling the corners with the original background keeps the image fully opaque, so the picture does not depend on 4D blending a transparent edge.

Blur radius in relation to detail spacing

A heavy blur over closely spaced detail averages that detail to a flat tone. The panel stops looking like glass and starts looking like plastic.

Real glass keeps large structure and loses fine detail only. Background content should be spaced widely enough to survive the blur, with contrast raised afterwards to restore what the blur flattened.

Saturation and contrast are one operation

Three steps of that procedure are the same arithmetic with different reference points.

Saturation moves each channel away from the pixel's own grey. That grey is the luma, a weighted average of the three channels. The formula is out = L + s x (C - L).

Contrast moves each channel away from a fixed tone instead. The formula is out = P + f x (C - P), where P is the pivot.

Both are the interpolation formula again, with a fraction greater than one. A fraction above one pushes a value away from the reference rather than towards it.

Table 6. The same expression, with the reference point changed.

Step	Reference point	Fraction	What it does
Blend a tint	the tint color	0 to 1	moves the color towards the tint

Saturate	the pixel's own luma	above 1	separates color from grey
Raise contrast	a fixed pivot tone	above 1	separates tone from the pivot
Blur	the neighbouring pixels	weights summing to 1	averages a neighbourhood

Recognising this keeps the implementation small. One expression, written once and checked once, performs every step in the table.

Where the technique stops, and what covers the rest

Table 7. What a pre-computed material delivers, and how the rest is covered.

Property of a real material	Delivered	How it is achieved
Background shows through	Yes	The blurred background is the panel's interior
Edge refraction	Yes	The edge band is resampled from a magnified copy
Specular rim	Yes	A one-pixel inlet above and shadow below
Saturation under the material	Yes	Applied before the tint
Samples the desktop behind the window	No	A fixed background stands in for the desktop, and reads the same in place
Changes as the window is dragged across the desktop	No	A pre-computed image is the same wherever the window sits
Refracts other interface objects	No	Those objects are drawn above the image, so the panel refracts the background only
Survives the window being resized	No	Each image is cut to one exact rectangle, so a panel that changes size would stretch it

A window of fixed size and position is unaffected by all four rows above. A design that needs the material to follow a moving desktop can add that later through a native plug-in.

The edge is cut after the blur

Order matters when a panel is both blurred and clipped.

Clipping first hands the blur a transparent surround. The blur averages that transparency inward, and the panel fades out across the width of the blur.

Cutting the edge after the filter keeps it crisp. The content being blurred then extends past the panel, so the blur finds paint on every side.

Note: a background that stops at the panel edge causes the same fault. The background rectangle is therefore drawn larger than the area it serves.

One operation, recurring in four roles

The interpolation introduced earlier is not confined to the color blend. Four steps of the procedure rest on the same formula.

Recognising that keeps the implementation small. One expression, verified once, serves every role.

Table 8. The same interpolation, in each of the roles the procedure requires.

Role	Interpolated between	The fraction used
Tint over a surface	The surface color and the tint color	The material alpha
The background field	Neighbouring heights on a coarse grid	The position inside one grid cell
Contour placement	The two corner heights of one cell	Where the contour level falls between them
Contrast recovery	A fixed pivot tone and the pixel	The contrast factor, greater than one

The fourth role uses a fraction greater than one. Values then move away from the pivot rather than toward it.

That is what raising contrast means. The pivot tone holds still and every other tone separates from it.

The second role weights its fraction before use. A straight fraction leaves the field piecewise linear.

Contours then follow the sampling grid and read as diamonds. A smooth weighting removes that artefact.

Note: Inside an image format that stores transparency, the drawing engine performs the blend. The arithmetic above is needed only because a form style sheet stores one flat color per property.

Contrast must be checked against the material

A color that reads well on an opaque background may disappear on a translucent one.

A small heading set in a tertiary label color is the usual casualty. Over a panel that blends to a similar tone, the words land within a few percent of the surface and vanish.

Text contrast should be judged against the surface that will actually sit behind it.

Responding to an appearance change

macOS can switch between light and dark appearance while an application is running. An open 4D window follows that change only in part. Some of its objects update and others keep the appearance they opened with.

A form resolves its style sheet once, when it opens

Form objects keep the values the style sheet gave them at open. Controls drawn by the system follow the new appearance immediately.

A window left open across an appearance change therefore shows both. Rectangles keep their original tones while list boxes and inputs follow the system.

Reading the style sheet once is a deliberate design. It is what keeps a drawn window stable and fast while the operator is using it.

Reopening as a strategy

The practical remedy is to open the window again. The form then resolves its style sheet against the new appearance.

The window should record the appearance it was built under. On Activate it compares that record with the current appearance.

A difference means the system changed while the window was not in front. Closing and reopening restores a consistent appearance.

```
: ($event.code=On Activate)
  var $now : Text:=String(Try(FORM Get color scheme))
  If (($now#"") & ($now#String(Form.scheme)))
     Form.reopen:=True
     CANCEL
  End if
```

Detecting an appearance change when the window returns to the front.

The change is not instant. It occurs when the window next becomes active, which is usually the moment the operator returns to it.

Window chrome, and how to reproduce it

Two window features common on macOS are assembled in the form rather than supplied ready-made. Both affect layout and are therefore decided early.

The unified title bar

Recent macOS applications place the window title, a subtitle and toolbar controls in one band at the top of the window.

4D draws the window title bar, and form objects are placed in the form's own area. The band is therefore built inside the form, where it can carry anything.

The practical answer is to draw a band inside the form, carrying the title, the subtitle, a search field and any action controls.

Note: A height taken from another application's window usually includes that application's title bar. A form built to that exact height produces a taller window once 4D adds its own.

Resizable split layouts

A three-pane window on macOS is normally resizable. The side panes keep their width while the middle pane absorbs the change.

4D can reproduce that behaviour. Each object carries `sizingX` and `sizingY` properties controlling how it responds to a resize.

The cost is per object. A window of sixty objects needs sixty decisions, and an object left at the default moves when it should stay.

Table 9. Common macOS window features, and how each is produced in 4D.

Feature	Produced by	How
Unified title bar with controls	The form	Draw a band inside the form, directly below the window title bar
Behind-window translucency	The form	Pre-compute a material over a fixed background
Resizable panes	Object properties	Set sizingX and sizingY on every object
System selection highlight	4D	Leave it alone, and it follows the operator's accent color
System appearance change while open	The form method	Reopen the window when it next becomes active
Animated transitions	The design	Section changes are drawn immediately

The fourth row has a consequence for color choice. A selection highlight left to 4D follows the accent color set by the operator. A fixed color does not.

Results

The techniques above were applied to a charter-management window. The window contains a navigation sidebar, a sortable register and a detail inspector.

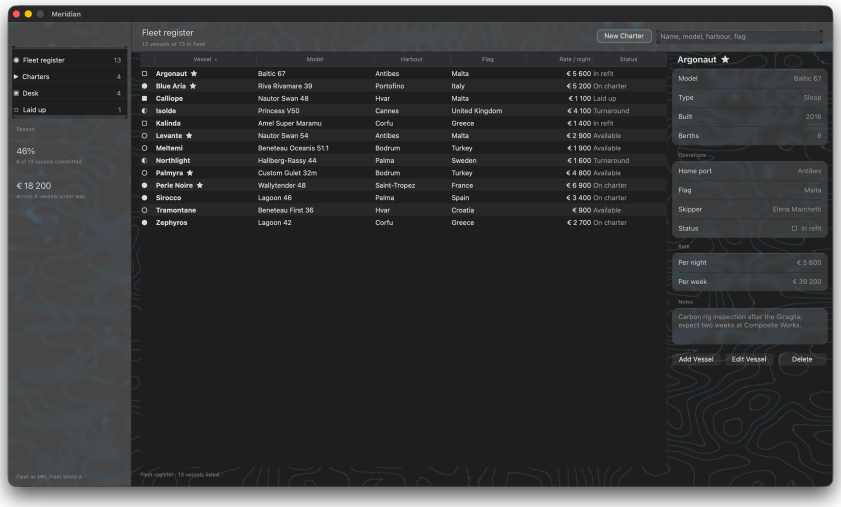


Figure 5. The finished window in dark appearance. The sidebar and the grouped panels are translucent. The register is opaque content.

Figure 6 shows the inspector. The background contours are sharp outside each panel and diffused within it, which is what reads as optical depth.

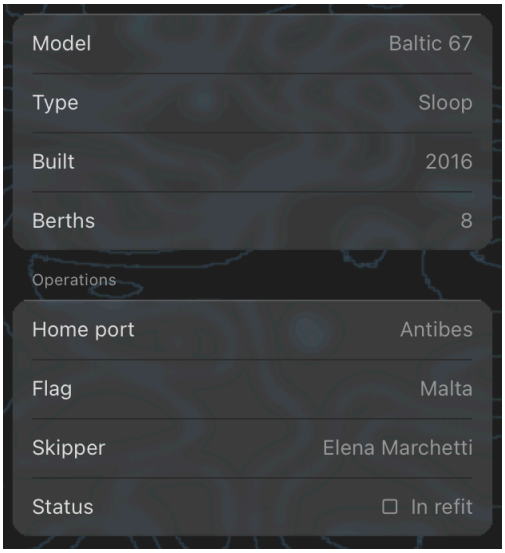


Figure 6. Grouped panels over a fixed background.

Figure 7 shows the navigation sidebar. The section list is transparent, so the panel behind it remains visible.

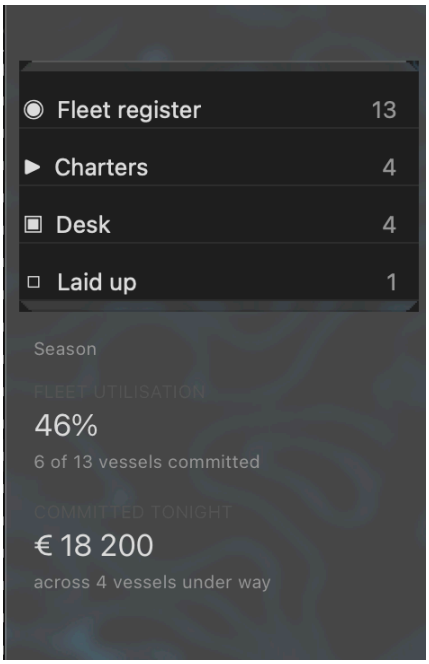


Figure 7. A transparent list box placed on a translucent panel.

Two details in Figure 7 were corrected only after looking at the window. Neither was visible in the form description file.



Figure 8. Confirm dialog.

- The list box originally painted an opaque background, hiding the panel behind it.
- The small headings originally used a tertiary label color, which disappeared against the panel.

Key Takeaways

Translucency is arithmetic

A translucent surface is a weighted average of two colors. Where an environment accepts a flat color, that average can be computed before the value is written.

Nothing is lost by computing it early. It is the same arithmetic a compositor would perform at draw time.

A fixed background can be solved once

Blur, refraction and sampling are costly mainly because the background usually moves. When it cannot move, what sits behind every panel is known in advance.

The optical work is then done once, before the application ships. What remains is an ordinary image, and drawing an image is something every form already does.

Store the alpha, not the result

One alpha serves both appearances, because only the tint changes between them.

Storing finished colors doubles the table and hides the single number that produced it.

A material needs something behind it

Translucent material has no color of its own. Over a flat ground it collapses into a plain rectangle.

Conclusion

None of this requires a plug-in, a browser, or a secondary programming language. Each item functions as a standard feature of 4D, applied strictly according to its native purpose.

Each component also delivers greater capabilities in combination than in isolation. The style sheet, the picture object, and the vector renderer serve as a clear demonstration of that principle.

The interface layer rewards precision rather than force. Stating intent with exactness, preparing every value prior to retrieval, and computing anything that cannot be rendered dynamically while the window is open remain essential practices.

These three habits sustain the material presented here and will support whatever is built next.